Part 5 Brief · CI eval gate and live evals

mcp-ops-server · 18 Oct 2026 · PR #6 · Status: Codex review fixed (4x P1, 2x P2); live run waits on your key

WHAT WAS BUILT

- **An eval gate on every push and PR:** a dedicated CI job runs all 31 golden tasks against the real server and all 12 negative controls. Free (no model, no secrets); any failure fails the build. The report lands in the job summary and as an artifact.
- **A live OpenAI agent** on the same tasks, graded by the same grader. Paid, so it runs only when you trigger it: Actions > Live evals > Run workflow, with the model ID as an input.
- **Live metrics:** pass rate, tokens, cost per task (from prices you pass), model and task latency p50/p99, turn-limit hits, and each final answer (shown, never graded).

HOW IT WORKS

Piece	Role
agents_openai.py	Chat Completions tool-calling loop. The model sees exactly the tools the server advertises, and every call goes through the harness's recorded call(), so it cannot grade itself.
ci.yml: evals	Replay agent + negative controls on every push and PR. Mark it a required check to block merges.
live-evals.yml	workflow_dispatch only. The key lives in a live-evals environment that only main can use (enforced by GitHub, not by this file). Inputs via env (no script injection), 30-min timeout.

WHY IT WAS DONE THIS WAY

- **Free gate, paid measurement.** Replay is deterministic, so it can block merges. A model's score varies run to run, so live runs report a number and do not fail on it unless you ask (`--min-pass-rate`). An API error does fail the run: it measured nothing.
- **Manual trigger only, main only.** No schedule and no PR trigger, so cost stays under your control, and code that has not been merged never runs with the key.
- **Optional dependency.** openai is a separate install extra imported lazily; the server and the CI gate never install it. Tests use a fake client and the real SDK against a local endpoint: no key.

WHAT WENT WRONG OR SURPRISED ME (LOGGED IN FAILURES.MD)

- **Part 4's evals never tested what a model reads.** The replay agent ignores tool descriptions, so a misleading description passed every eval. The harness now hands each agent the server's own tool list; the live agent is scored on it.
- **Codex review: 4x P1, 2x P2, all fixed with tests.** The real client broke from the 2nd task on (a loop per task; my fake client had no connection pool to break). Malformed tool calls escaped the grader. A dispatch on an unreviewed branch could reach the key. A run where every API call failed was green. Also: partial token usage lost on error, and a prompt rule that contradicted a task.

VERIFY IT YOURSELF

```
python -m evals.run       # 31/31 tasks passed; 12/12 negative controls caught
ruff check . && python -m pytest -q   # 170 passed (11 for the live agent loop)
PR #6 > Checks > evals > Summary      # the report CI just produced
```

INTERVIEW TALKING POINT

"Every PR runs a free, deterministic trajectory eval that blocks merges. Real-model runs are a separate, owner-triggered measurement of pass rate, cost and latency, graded by the same code, and only merged code on main can reach the API key."

NEXT

Your actions: mark **evals** a required check; create environment **live-evals** limited to main and add OPENAI_API_KEY there (not as a repo secret), plus an OpenAI spending limit; trigger Live evals with your model ID. I then turn the first live run into the README's results table. Part 6 waits for your "go".